

API Security & Performance Testing using JMeter and Jira

Name: W.A.D. Chaduki Amashi

Degree: Bachelor of Software Engineering (Hons)

University: The Open University of Sri Lanka

Date: 22/04/2026

Introduction

This project focuses on security and performance testing of APIs. The aim was to identify vulnerabilities, analyze system behavior under load, and ensure reliable API responses.

Security testing was performed to check issues such as unauthorized access, input validation, and data exposure. Performance testing was conducted to evaluate system performance using multiple user requests.

Objectives

- Identify security vulnerabilities
- Perform performance testing under different load conditions
- Report defects using Jira
- Analyze system behavior and response times

Tools & Technologies

Tool	Purpose
Postman	API Testing
Apache JMeter	Performance Testing
Jira	Bug Tracking
JSONPlaceholder API	API Testing (for security validation)

Scope of Testing

Testing focused on:

- API Security Testing
- API Performance Testing
- Bug tracking using Jira

Test Approach

The testing process included:

- Security testing was performed using Postman by sending valid and invalid inputs.
- Performance testing was conducted using Apache JMeter with multiple user loads.
- Measured response time, throughput, and error rate.
- Identified issues were documented and tracked using Jira.

Security Testing

Security testing was conducted to identify vulnerabilities in the API and ensure that the system handles invalid or malicious inputs securely. The focus was on unauthorized access, input validation, and data exposure.

Testing was performed using Postman.

Test Approach

- Sent API requests with valid, invalid, and malicious inputs
- Tested endpoints without authentication
- Checked system behavior for unexpected inputs
- Verified whether sensitive data is exposed

Security Test Cases

Test ID	Test Scenario	Test Case Description	Expected Result	Actual Result	Test Status
ST01	Invalid Endpoint	Verify API with invalid endpoint	returns 404 Not Found for invalid endpoint	API returned 404 Not Found error	Pass
ST02	Unauthorized Access	Verify API requires authentication	requires authentication before returning user data	Data returned successfully	Fail
ST03	Invalid Input	Verify API with invalid ID	return error response for invalid input	returned 404 Not Found error	Pass
ST04	Invalid Method	Verify unsupported HTTP method	reject unsupported HTTP methods	returned 404 Not Found error	Pass
ST05	Large Input Handling	Verify API with very large input	handle large input safely and return an error if invalid	returned 404 Not Found error	Pass
ST06	SQL Injection	Verify API against SQL injection	prevent SQL injection and not expose data	returned 404 Not Found and did not process the SQL injection input	Pass
ST07	Data Exposure	Verify sensitive data exposure	Only necessary data should be exposed to the user	API exposes email, phone and user details without restriction	Fail

Key Findings

- API allows access without authentication
- Sensitive data is exposed in responses

Conclusion of Security Testing

The security testing identified several vulnerabilities that could lead to data exposure and misuse. Proper authentication, validation, and security controls are required to improve system security.

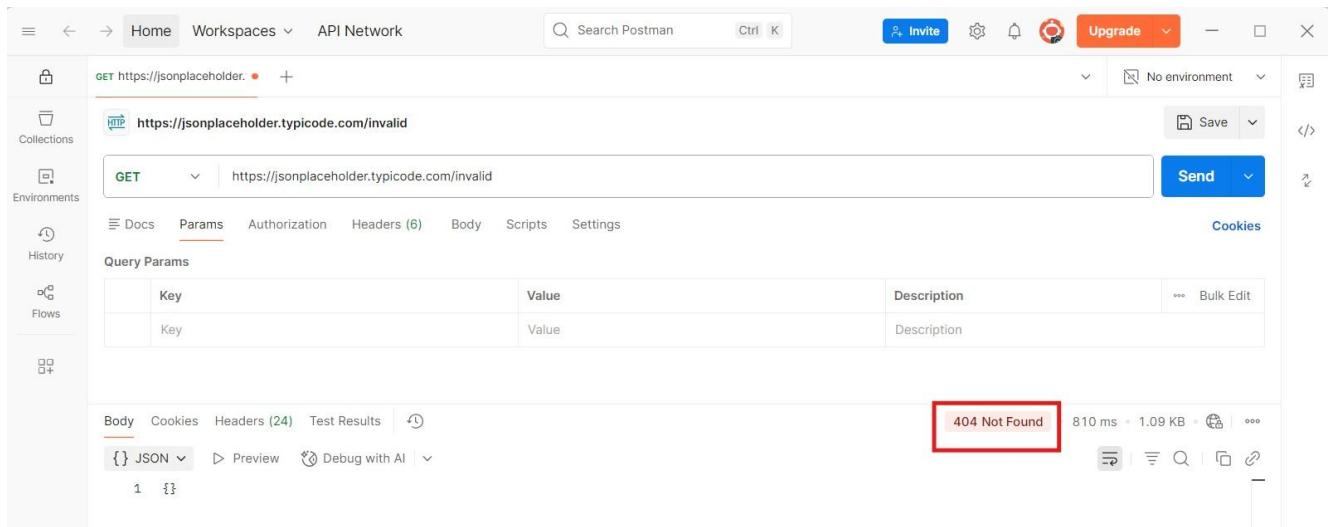


Figure 1: Verify API with invalid endpoint

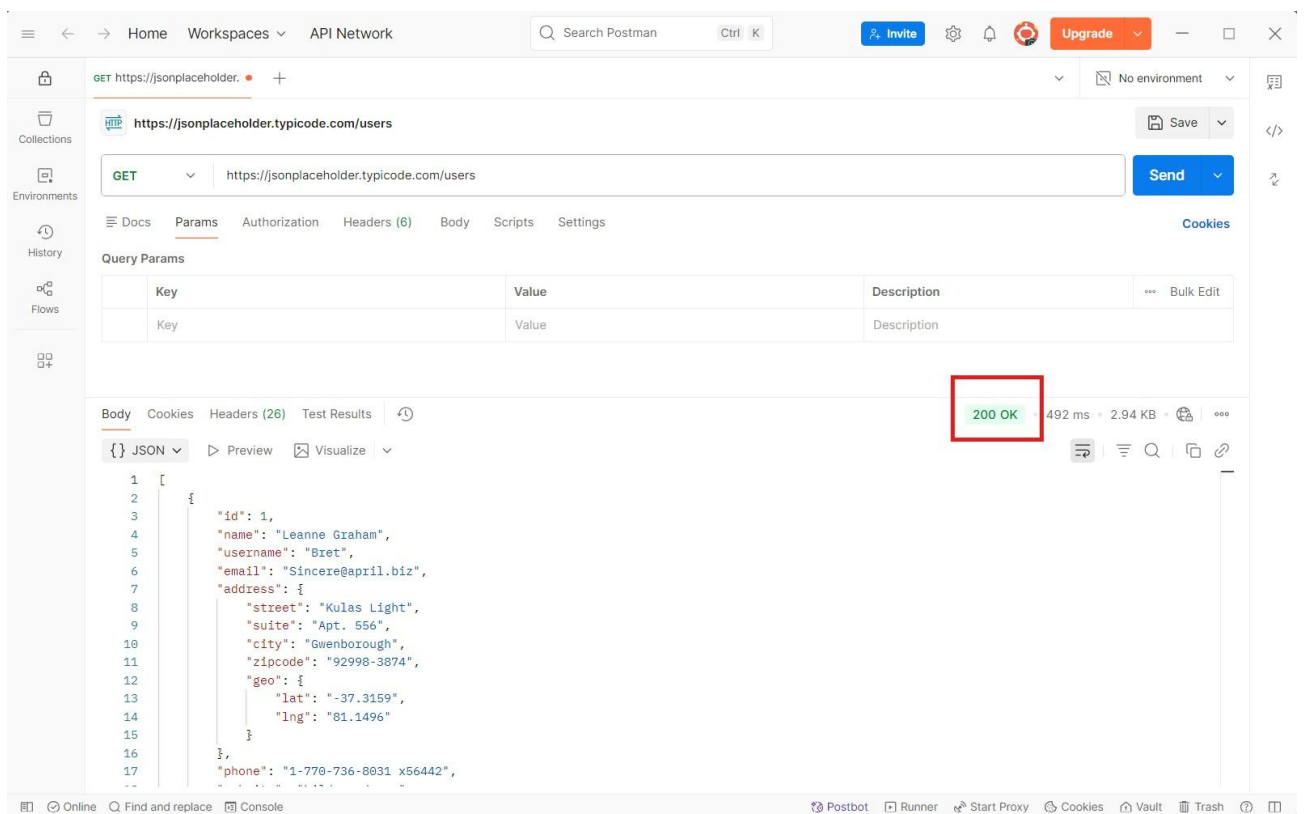
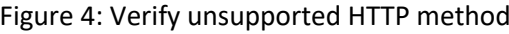


Figure 2: Verify API requires authentication



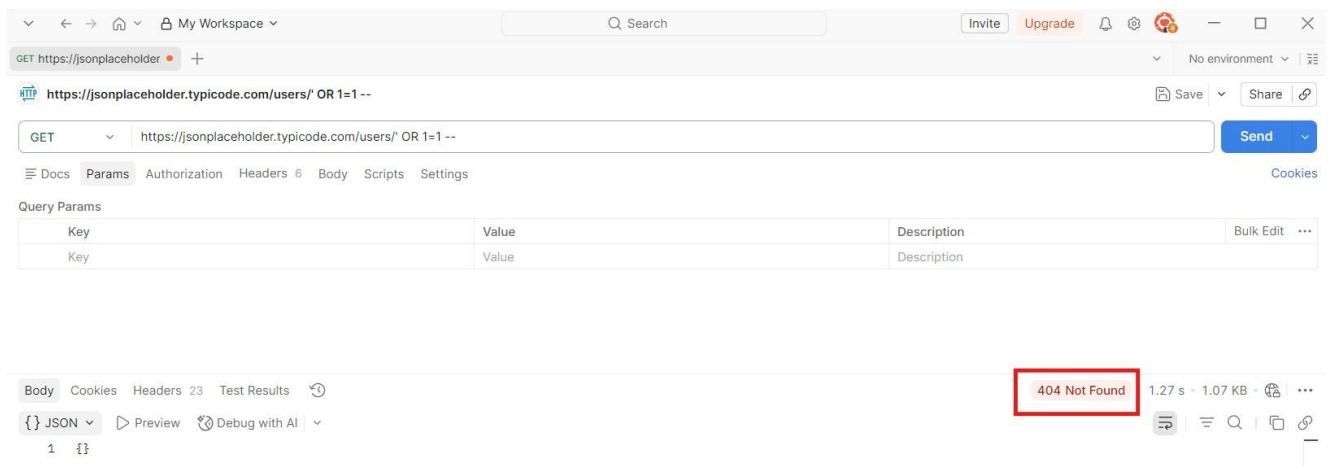


Figure 6: Verify API against SQL injection

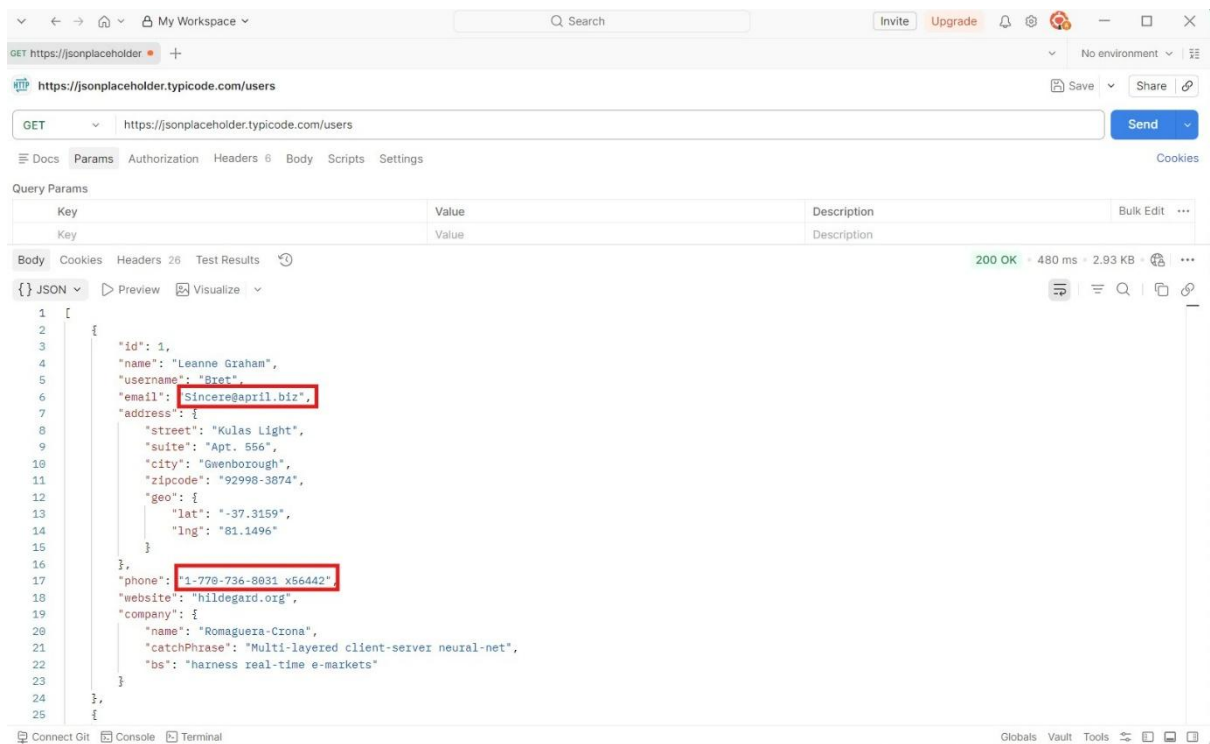


Figure 7: Verify sensitive data exposure

Defect Reporting (Jira)

A total of 2 high-severity defects were identified during testing, which could impact data security and system reliability.

Bug 1

Title: Unauthorized Access

Severity: High

Description: API returns user data without authentication

Bug 2

Title: Data Exposure

Severity: High

Description: API exposes email, phone and user details without restriction

Test Execution Summary

Total Test Cases	Passed	Failed
7	5	2

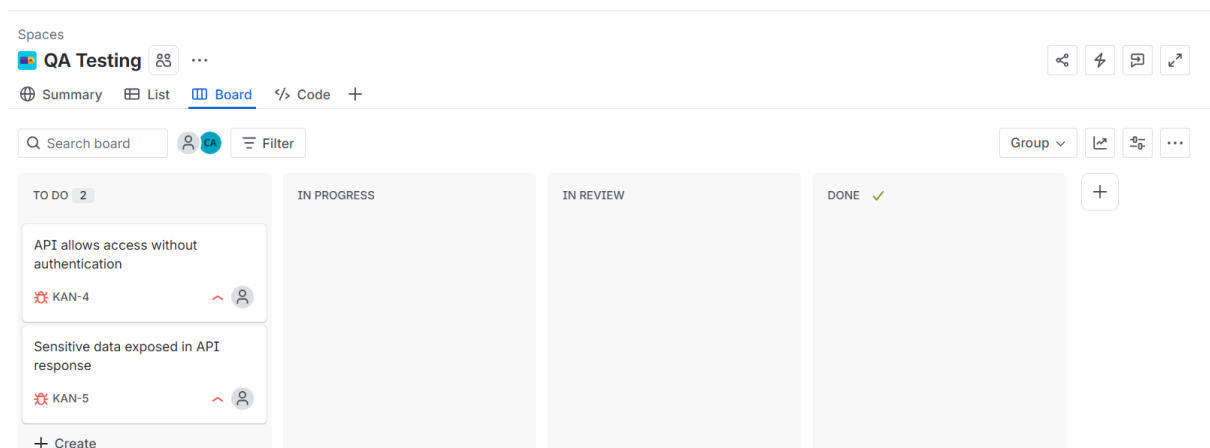


Figure 8: Jira board showing defects tracked during testing

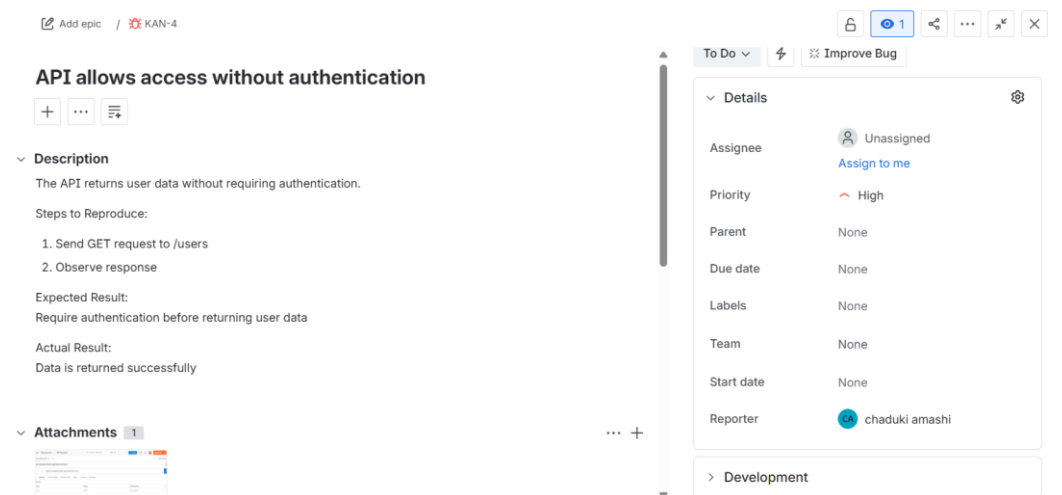


Figure 9: Detailed view of a reported bug in Jira

Performance Testing

Performance testing was conducted to evaluate how the API behaves under different user loads and to measure system performance.

Testing was performed using Apache JMeter.

Test Configuration

- Number of Users (Threads): 10, 50, 100
- Ramp-up Time: 5–20 seconds
- Loop Count: 1

Test Execution

Multiple tests were conducted by increasing the number of users to analyze system performance under load.

Results Summary

Users	Ramp-up Time	Avg Response Time	Error %	Throughput
10	5 sec	781 ms	0%	2.1/sec
50	10 sec	620 ms	0%	4.9/sec
100	20 sec	645 ms	0%	26.9/sec

Response time varied with increasing user load. The system performed efficiently under moderate load, with slight variations observed at higher user levels.

Conclusion of Performance Testing

The API handled moderate traffic efficiently, but performance degradation was observed as user load increased.

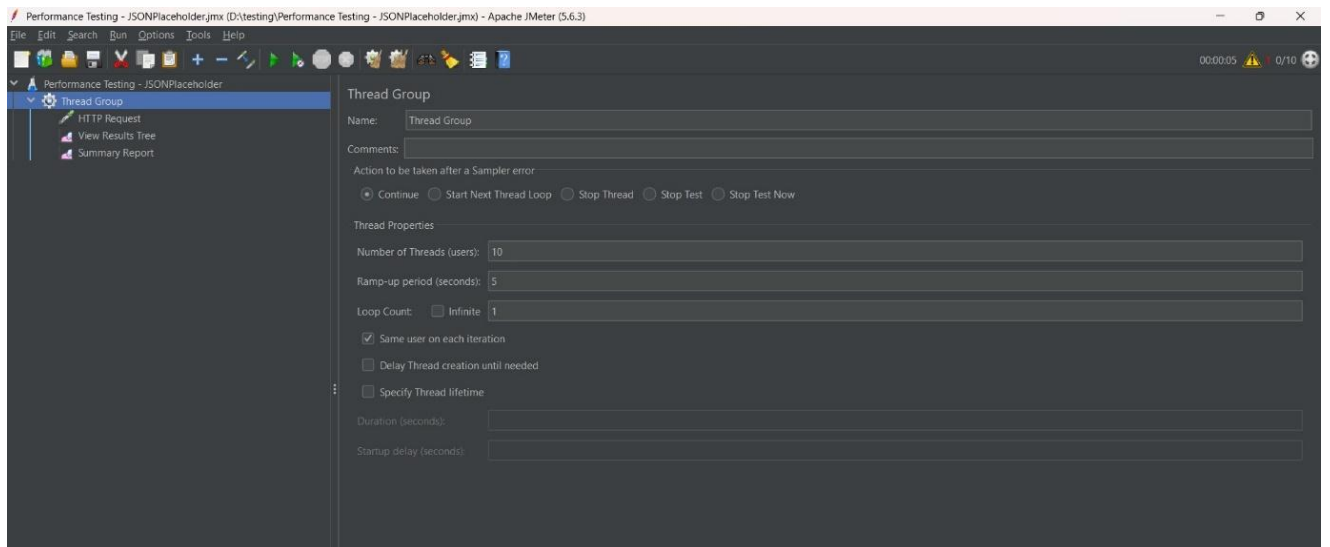


Figure 10.1: 10 users

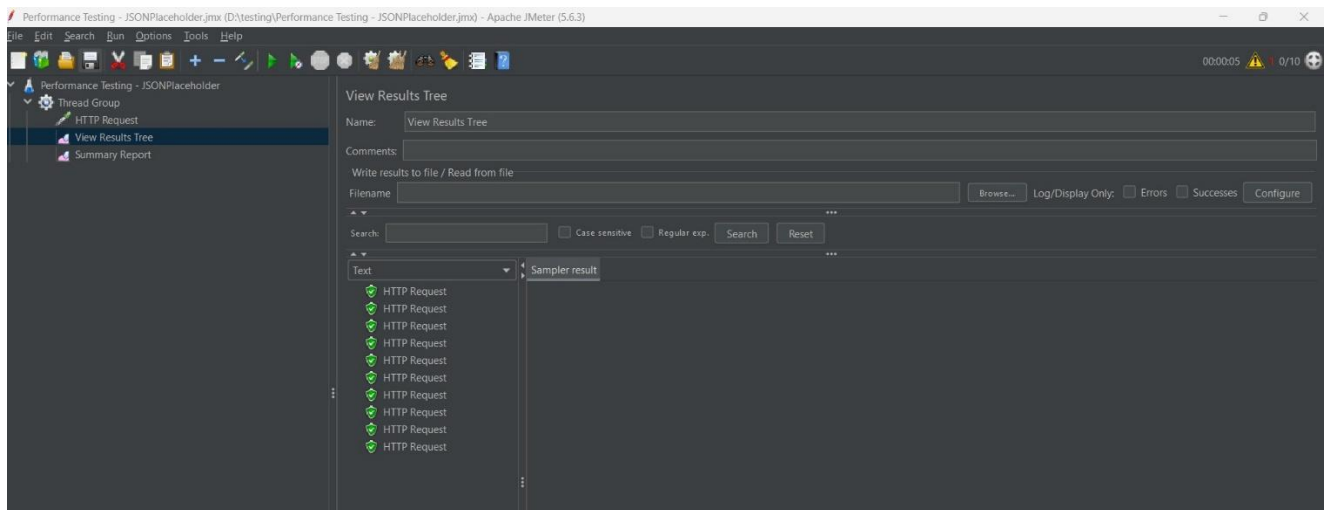


Figure 10.2: results tree

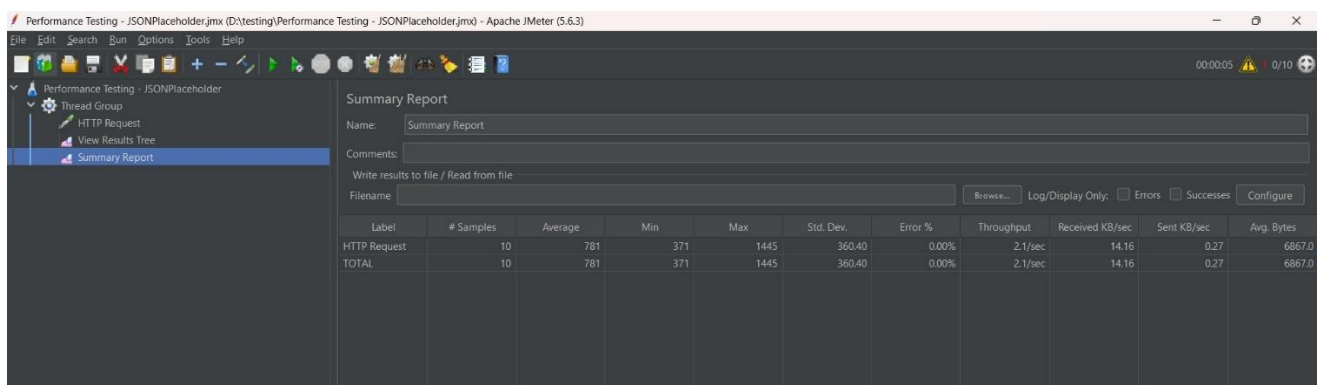


Figure 10.3: summary report

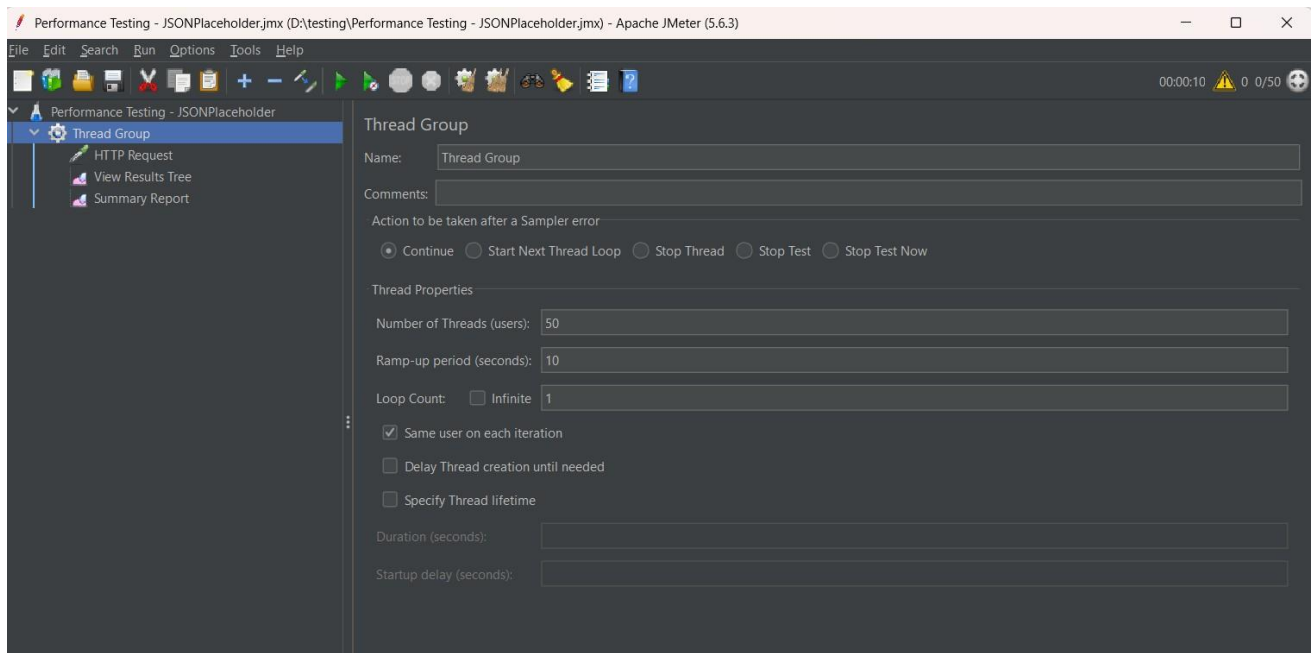


Figure 11.1: 50 users

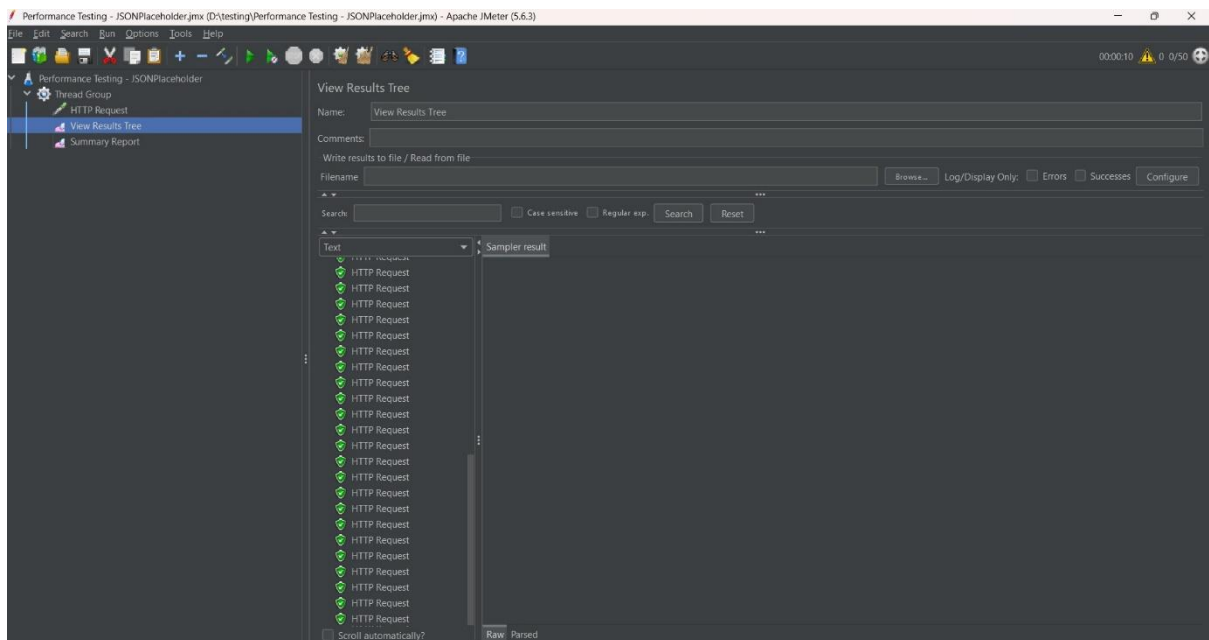


Figure 11.2: results tree

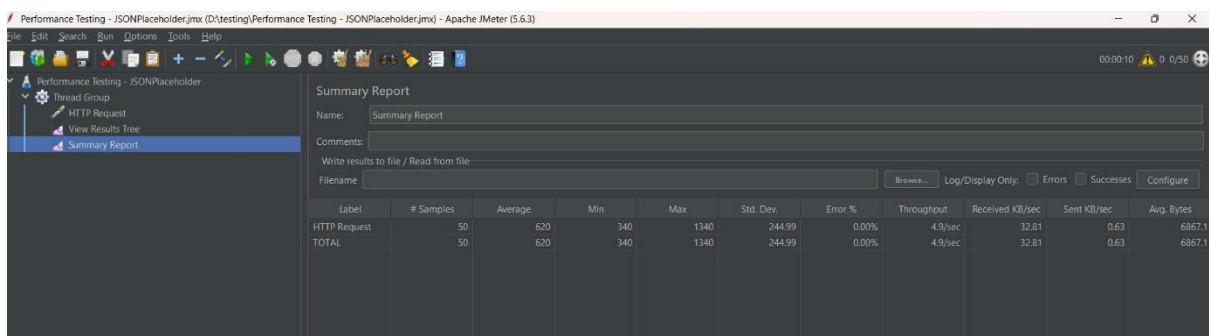


Figure 11.3: summary report

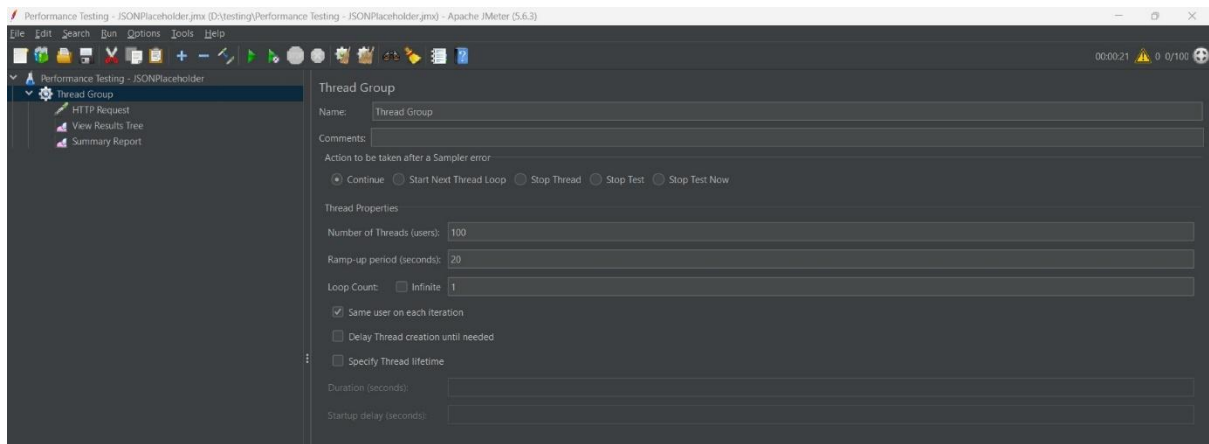


Figure 12.1: 100 users

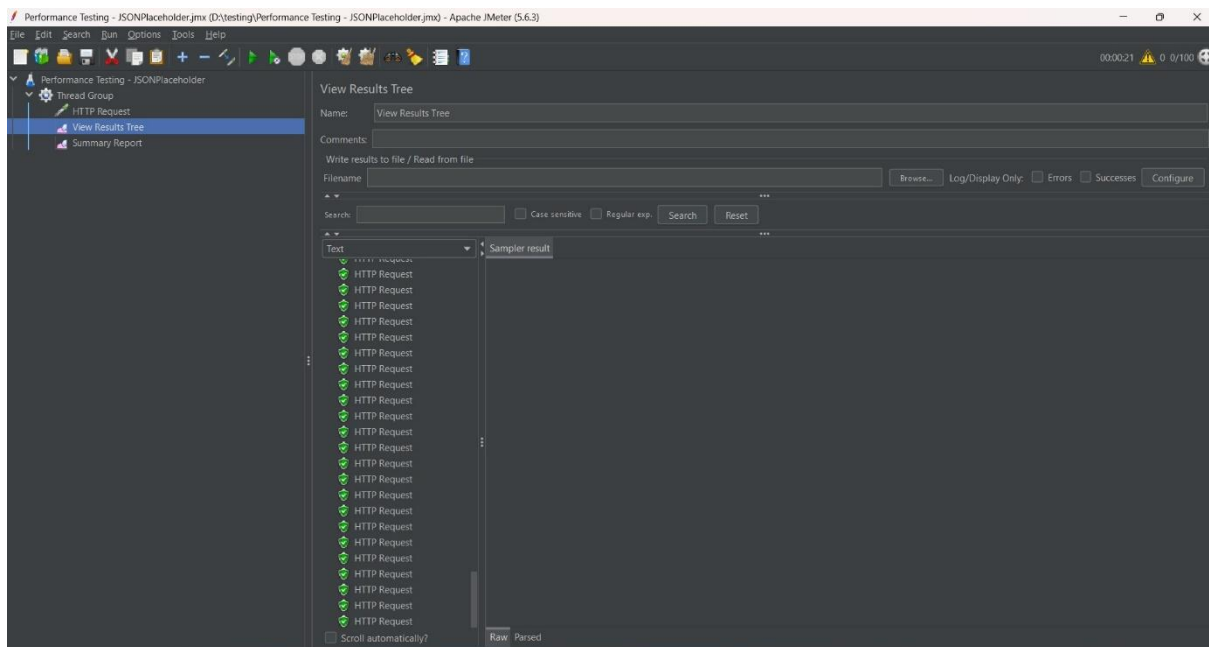


Figure 12.2: results tree

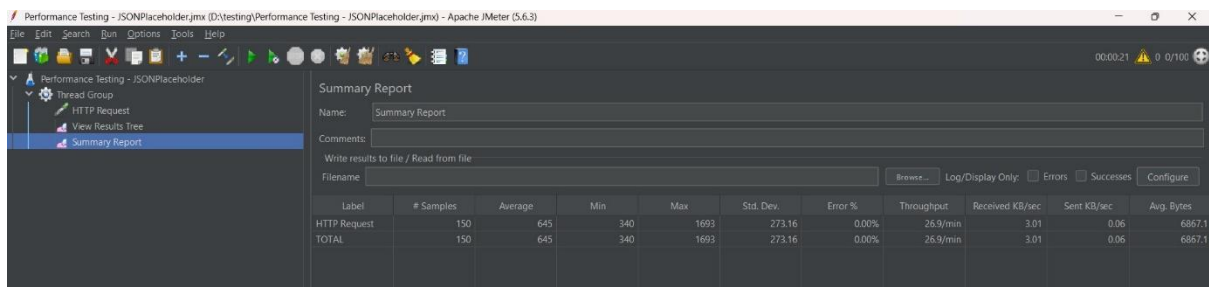


Figure 12.3: summary report

Challenges Faced

- Understanding basic concepts of API security testing and identifying vulnerabilities.
- Learning to use Postman effectively for sending different types of requests.
- Setting up and using Apache JMeter for performance testing.
- Analyzing performance metrics such as response time and throughput.
- Learning how to use Jira for bug tracking and issue management.

Conclusion

The project successfully evaluated the security and performance of APIs. Several vulnerabilities were identified, including lack of authentication and data exposure issues. Performance testing showed that the system handled moderate user load efficiently, with minor variations in response time at higher loads.

This project provided practical experience in security testing, performance testing, and defect tracking using industry tools.

Future Improvements

- Perform advanced security testing techniques.
- Increase performance testing with higher user loads.
- Improve test coverage by including more negative and edge test cases.
- Integrate testing into CI/CD pipelines for continuous testing.